

Scheduling

The Scheduler

The **scheduler** is the operating system component that allocates CPU resources to threads. It decides **which thread** runs on **which CPU**, **when** and for **how long**.

The **scheduling algorithm** (or **policy**) used to take these decisions depends on:

- The system itself, *i.e., how many CPUs, how much memory, etc.*
- The workloads running on it, *i.e., interactive applications, long running programs, etc.*
- The goals to achieve, *i.e., responsiveness, energy, performance, etc.*

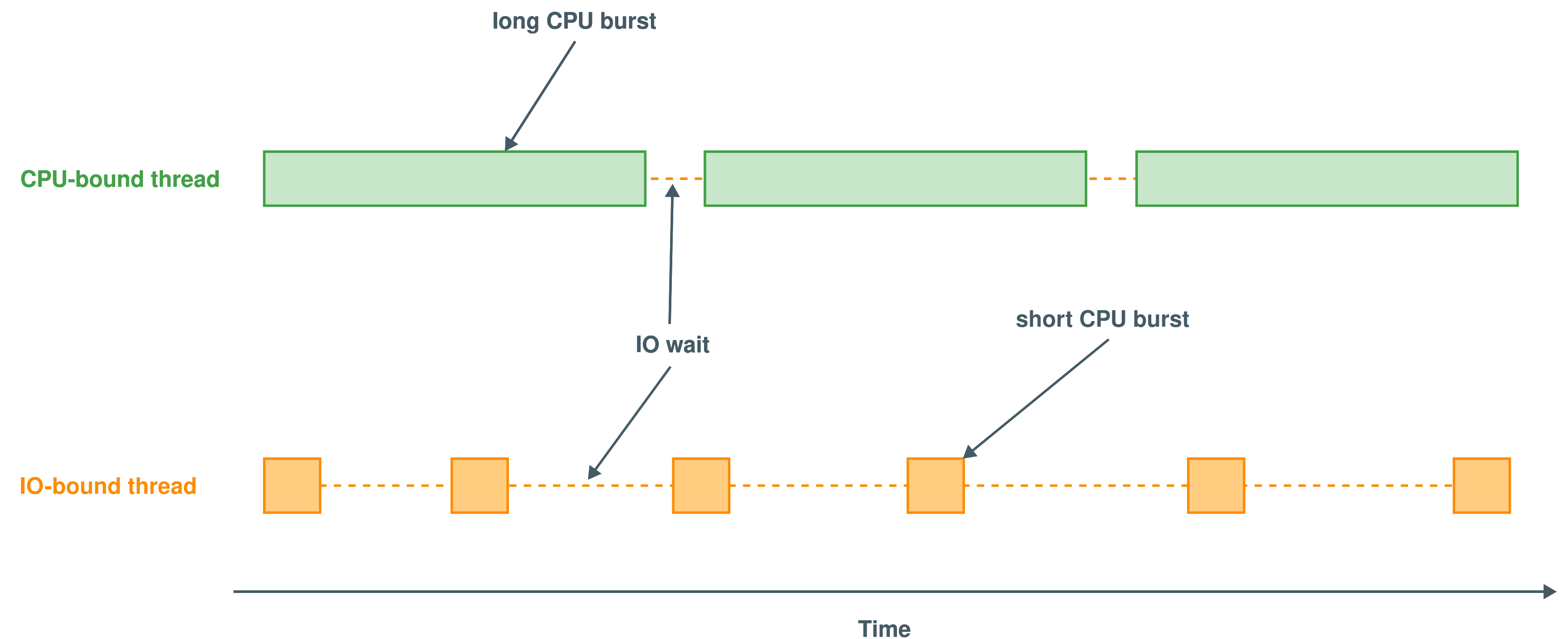
Application Behavior

Each application has its own behavior depending on what it does.

But most of them alternate computations (on the CPU, **ready** or **running**) and waiting on IOs (**blocked**), e.g., *waiting for data from disk*.

There are two main classes of applications:

- **CPU-bound** (or *compute-bound*): long computations with a few IO waits
e.g., data processing applications
- **IO-bound**: short computations and frequent IO waits
e.g., web server forwarding requests to worker threads



Triggering the Scheduler

Depending on the applications' and the hardware's behaviors, the scheduler decides which thread runs on which CPU.

Application behavior:

1. When a process/thread is created ($\emptyset \rightarrow \text{ready}$), i.e., after a `fork()/pthread_create()`
 $\implies$ Does the parent or child thread run first?
2. When a thread terminates ($\text{ready/running} \rightarrow \text{X}$)
 $\implies$ Which thread runs next?
3. When a thread waits on for an IO to complete ($\text{running} \rightarrow \text{blocked}$), e.g., waiting on a keyboard input
 $\implies$ Which thread runs next?

Hardware behavior:

4. An IO completes, triggering an interrupt and waking a thread ($\text{blocked} \rightarrow \text{ready}$), e.g., data request from the disk is ready, network packet arrival, etc.
 $\implies$ Let the current thread run or replace it with the waking thread?
5. A clock interrupt occurs
 $\implies$ Preemption?

Preemption

An important characteristic of a scheduler is whether it is preemptive or not.

A **preemptive** scheduler can forcefully interrupt a running task to execute a different one.

In other words:

- **Non-preemptive** schedulers choose a thread to run and let it run until it blocks, terminates or voluntarily yields the CPU
- **Preemptive** schedulers might stop a running thread to execute another one after a given period of time

Note

Preemption is only possible on systems with a hardware clock.

Categories of Scheduling Algorithms

Depending on the system, workloads and goals of the system, different categories of algorithms should be used:

- **Batch:** Business applications, with long running background tasks (compute-bound)
e.g., accounting, file conversion/encoding, training machine learning models, etc.
—→ Usually no preemption to minimize scheduling overhead
- **Interactive:** Users are interacting with the system, or IO-bound applications
e.g., text editor, web server, etc.
—→ Preemption is **necessary** for responsiveness
- **Real time:** Constrained environments, where deadlines must be met
e.g., embedded systems
—→ May or may not have preemption, as these systems are usually fully controlled, with no arbitrary programs running

Scheduling Metrics

Depending on the workloads, different goals are desirable for the scheduling algorithm. In other words, the scheduler can optimize different metrics depending on requirements.

All categories

- **Fairness:** Processes should get a similar amount of CPU resource allocated to them. **Fairness is not equality**, the amount of CPU allocation can be weighted, *e.g., depending on priority*
- **Resource utilization:** Hardware resources, *e.g., CPU, devices*, should be kept busy as much as possible

Batch

- **Throughput:** Maximize the number of jobs completed per unit of time, *e.g., packets per second, MB/s, etc.*
- **Turnaround time:** Minimize the duration between job submission and completion
- **CPU utilization:** Keep the CPU busy as much as possible

Interactive

- **Response time/latency:** Respond to requests quickly
- **User experience:** The system should *feel* responsive to users

Real time

- **Meeting deadlines:** Finish jobs before the requested deadlines to avoid safety problems
- **Predictability:** Behave in a determinist manner, easy to model and understand

Now, let's see some scheduling policies!

Batch Scheduling: First-Come, First-Served

The **First-Come, First-Served (FCFS)** scheduler assigns threads to CPUs in their order of arrival, in a non-preemptive way.

Runqueue: Threads are stored in a list sorted by order of arrival, with the first element being the *oldest*

Election: Choose the head of the runqueue

Block: Remove the thread from the runqueue and trigger an election

Wake up: Add the thread to the runqueue (at the end, like a newly created thread)

Event	Running	Runqueue
<i>Initial state</i>	$\emptyset$	$A \rightarrow B \rightarrow C \rightarrow D$
<i>Election</i>	A	$B \rightarrow C \rightarrow D$
<i>A waits on an IO</i>	B	$C \rightarrow D$
<i>A wakes up</i>	B	$C \rightarrow D \rightarrow A$
<i>B terminates</i>	C	$D \rightarrow A$
<i>E starts</i>	C	$D \rightarrow A \rightarrow E$

+ Easy to implement

– With many threads, long latencies for IO-bound threads and high turnaround time for CPU-bound threads

Batch Scheduling: Shortest Job First

The **Shortest Job First (SJF)** scheduler assumes that a job's run time is known in advance. It works in the same way as FCFS, except that the job queue is sorted by run time.

Runqueue: Threads are stored in a list sorted by increasing total duration

Election: Choose the head of the runqueue

Block: Remove the thread from the runqueue and trigger an election

Wake up: Add the thread to the runqueue (at its sorted position)

$$\text{Average turnaround time} = \frac{4a+3b+2c+d}{4}$$

Let's assume four tasks A , B , C and D , arriving in that order and the following known durations:

Policy					Total turnaround	Average turnaround
FCFS	A (5)	B (3)	C (4)	D (3)	15	10
SJF	B (3)	D (3)	C (4)	A (5)	15	8.5

+ Optimal when all jobs are known in advance

– Does not adapt well to differed job arrivals

Batch Scheduling: Shortest Remaining Time

The **Shortest Remaining Time (SRT)** scheduler is a modified **preemptive** version of SJF, where jobs are sorted by their **remaining** run time.

Runqueue: Threads are stored in a list sorted by increasing remaining duration

Election: Choose the head of the runqueue

Block: Remove the thread from the runqueue and trigger an election

Wake up: Add the thread to the runqueue (at its sorted position). If it has a lowest remaining time than the running thread, preempt it

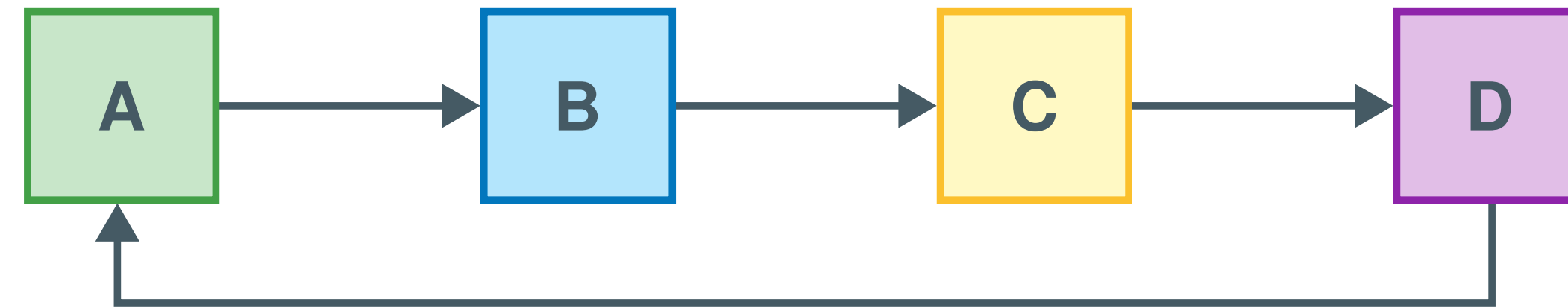
Time	Running	Runqueue	Event
init	$\emptyset$	$B(1) \rightarrow C(4) \rightarrow A(5)$	
0	$B(1)$	$C(3) \rightarrow A(5)$	
1	$C(4)$	$A(5)$	
2	$D(1)$	$C(3) \rightarrow A(5)$	$D(1)$ arrives
3	$C(3)$	$A(5)$	
6	$A(5)$	$\emptyset$	

Interactive Scheduling: Round-Robin

Round robin is a simple **preemptive** scheduler.

Each thread is assigned a time interval (*quantum*) during which it is allowed to run.

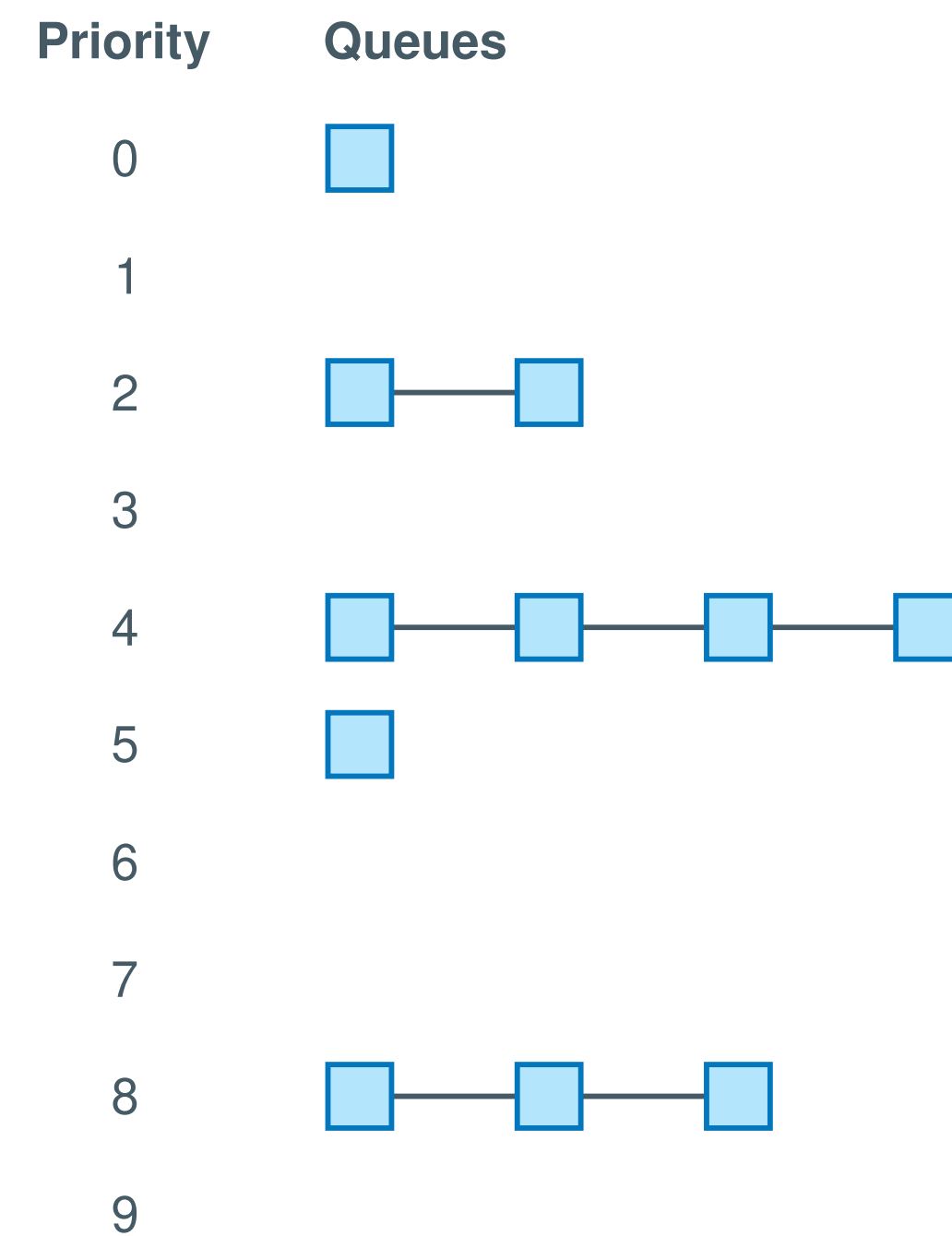
When a thread uses up its quantum, it is preempted and put back to the end of the queue.



Interactive Scheduling: Priority-based

Each process is assigned a priority, and the **priority-based** scheduler always runs the thread with the highest priority. Priorities might be decided by users or by the system depending on the thread behavior (IO- or compute-bound).

Example: **Multilevel queueing** (or MLQ).



Multilevel feedback queueing

The system may change priorities to avoid threads hogging the CPU or starvation, and assign a quantum to threads to improve fairness.

Interactive Scheduling: Fair

A **fair** scheduler tries to give the same amount of CPU time to all threads.

Some may also enforce fairness between users (so-called *fair-share* schedulers),
e.g., user A starts 9 threads, user B starts 1 thread, they should still get 50% of the CPU each.

Interactive Scheduling: Lottery

A **lottery** scheduler randomly chooses the next thread running on the CPU.
The randomness can be skewed by giving more *lottery tickets* to higher priority threads.

Real Time Scheduling

Real time schedulers must enforce that threads meet their deadlines.

They can be separated in two classes:

- **Hard real time:** Deadlines **must** be met or the system breaks
e.g., the brake system of a car must respond to the pedal being pressed very quickly to avoid accidents.
- **Soft real time:** Deadlines can be missed from time to time without breaking the system
e.g., a video player must decode 50 frames per second, but missing a frame from time to time is fine.

Real time applications may also be:

- **Periodic:** the task periodically runs for some (usually short) amount of time,
e.g., the frame decoder of the video player is started every 20 ms and must complete in less than 5 ms.
- **Aperiodic:** the task occurs in an unpredictable way,
e.g., the brake mechanism of a car is triggered when the pedal is pressed and must engage the brakes in less than 1 ms.

Examples of real time schedulers: Earliest Deadline First (EDF), Deadline Monotonic, etc.